Active Object

Active Object · Serialize ledger mutations on one servant · Build with Waqar

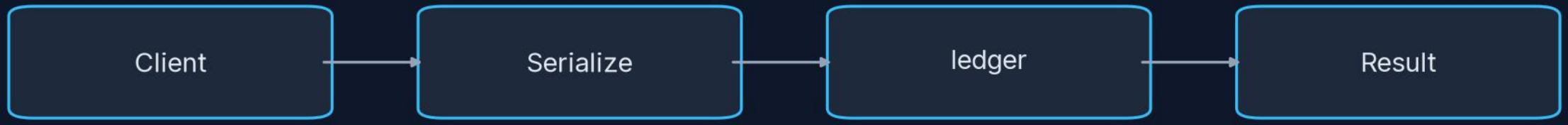
Today's Hook

- Problem: Serialize ledger mutations on one servant
- Pattern: Active Object — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Serialize ledger mutations on one servant
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Active Object: Serialize ledger mutations on one servant

Real-World Scenario

- Running example: Serialize ledger mutations on one servant
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.concurrency.activeobject;

import com.buildwithwaqar.patterns.concurrency.activeobject.ActiveObject;
/** Problem: serialize ledger mutations without external locks in callers. */
public final class UseCaseDemo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Active Object problem: serialized ledger ===");
    try (var ao = new ActiveObject()) {
      ao.add(10).get(); ao.add(-3).get();
      System.out.println("balance=" + ao.get().get());
    }
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.concurrency.activeobject;

import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

/** Active Object — all calls run on a dedicated async servant thread. */
public final class ActiveObject implements AutoCloseable {
    private final ExecutorService servant = Executors.newSingleThreadExecutor(r -> {
        var t = new Thread(r, "active-object");
        t.setDaemon(true);
        return t;
    });
    private int value;

    public Future<Integer> add(int delta) {
        return servant.submit(() -> { value += delta; return value; });
    }

    public Future<Integer> get() {
        return servant.submit((Callable<Integer>) () -> value);
    }
```

Solution (2)

```
    @Override public void close() { servant.shutdownNow(); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.concurrency.activeobject;

import com.buildwithwaqar.patterns.concurrency.activeobject.ActiveObject;
/** Problem: serialize ledger mutations without external locks in callers. */
public final class UseCaseDemo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Active Object problem: serialized ledger ===");
    try (var ao = new ActiveObject()) {
      ao.add(10).get(); ao.add(-3).get();
      System.out.println("balance=" + ao.get().get());
    }
  }
}
```

Walkthrough Output

```
=== Active Object problem: serialized ledger ===
balance=7
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Active Object.

Related Patterns

- Contrast Active Object with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Active Object → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.